

tiptotyp

Code review & architecture audit

A source-based assessment of correctness, ownership, repeated logic, dead code, and credible ways to reduce the codebase.

38,090

Rust physical lines

145

fields in EditorApp

383

passing test executions

Assessment

The codebase has useful domain modules and substantial deterministic coverage. Its main weakness is that `EditorApp` still owns too many independent lifecycles. Rendering, document mutation, sidecar coordination, settings application, asynchronous dialogs, and native child windows are joined through manually synchronized fields.

The largest improvement is **explicit ownership of document, build, presentation, and workflow state**. Moving functions into more files is a useful first step, but meaningful LOC reduction comes from replacing repeated policy and state transitions with shared implementations.

Decisions recommended

- Fix the failure paths first: gallery rollback, artifact/raster coupling, private-workspace races, theme calculations, and launch/persistence boundaries.
- Keep the existing Rust/egui application and its two preview backends. Refactor by subsystem instead of rewriting the application.
- Budget approximately **1,100–2,100 net production/tooling lines** for an initial consolidation program, subject to implementation spikes. Do not count relocated code or deleted tests as savings.

Scope: the current repository checkout, recorded at commit `10bd0295c066e7866a066c115f9e76e3a7f9be63`. Report only; application source was not edited.

Scope and evidence

The review covers all 28 Rust application modules, three integration-test files, the packaging task, the gallery script, and the relevant architecture and QA documents. Source references below use repository-relative paths and line numbers from the recorded checkout.

At the start, `README.md`, `app.rs`, `compiler.rs`, `search.rs`, and `tinymist.rs` had local edits. Those edits were included in the review. During the audit they appeared in commit `10bd029`; recorded Rust file hashes remained unchanged. No source changes or repairs were made as part of this report.

How to interpret findings

P1 means a failure path capable of losing existing output and deserving immediate attention. **P2** means a user-visible correctness, reliability, or substantial latency problem. **P3** means structural debt or cleanup to schedule with related work. These priorities assess the described trigger, not the likelihood of every user encountering it.

Reproduced means an isolated executable probe demonstrated the behavior. **Static** means the source and call path establish the issue, but the whole application interaction was not exercised. **Architecture** identifies a concrete organizing opportunity, not a proven runtime defect.

Required checks

Command	Result
<code>cargo fmt --all -- --check</code>	Pass
<code>cargo clippy --all-targets -- -D warnings</code>	Pass on this macOS checkout
<code>cargo test --no-fail-fast</code>	376 passed; 2 ignored
<code>cargo test --manifest-path xtask/Cargo.toml</code>	7 passed

The 376 application/integration test executions include repeated inline tests compiled through path-included modules. They are not 376 independent behaviors. The ignored tests require a real Typst watcher/Poppler environment and a real Tinymist executable respectively.

Limits

This was a static review with targeted probes, not a fuzzing campaign, penetration test, benchmark suite, or native UI acceptance run. Windows and Linux execution, cross-volume writes, full native composition, and real-user preference persistence were not exercised. No application screenshots were needed for a report-only change. Report PDF pages were separately rendered and inspected; that is document QA, not application visual verification.

LOC means physical lines, including whitespace and comments. “Before test module” means the lines preceding the final top-level `#[cfg(test)] mod tests`; it is a useful production-size proxy, not a lexical code-only measure. Estimates are engineering judgments and can be offset by better error handling and tests.

Architecture as it stands

Process shell (`windowing.rs`)

Owns document viewports, shared settings, active-session routing, capture batches.

Per-window editor (`app.rs`)

Owns buffer + history + save workflow + workspace + compiler + Tinymist + assets + native preview + settings UI + popups + hover + screenshots.

Services and domain helpers

Compiler → private mirror → Typst watcher → PDF → Poppler → page pixels.

Tinymist → LSP → localhost preview → Wry child webview.

Syntax / search / diagnostics / project index / filesystem / themes / fonts.

Where the size is concentrated

Module	Total	Before tests	Dominant responsibility
<code>app.rs</code>	15,643	13,370	UI and session orchestration
<code>tinymist.rs</code>	3,227	2,423	Protocol and process lifecycle
<code>theme.rs</code>	2,030	1,588	Visual tokens, fonts, styles
<code>screenshot.rs</code>	1,959	1,225	Capture, launch, scene contract
<code>builtin_themes.rs</code>	1,469	1,127	Mostly intentional theme data
<code>compiler.rs</code>	1,255	1,020	Watcher and raster rendering
<code>highlight.rs</code>	1,194	841	Parsing and display decoration

`app.rs` contains 41.1% of measured Rust lines and 45.6% of the lines before inline test modules. Its largest methods include `show_settings` (561 lines), `show_editor` (363), `new_session` (266), and `apply_snapshot_scene` (215). Size identifies a review hotspot; the repeated ownership decisions below explain why it matters.

Foundations to preserve

The Typst syntax parser, normalized diagnostics, Unicode-aware editing, private file staging, latest-generation rejection, pure preview geometry, and observable backend fallbacks already form useful boundaries. The packaging task checks archive hashes and member types before streaming allowlisted files. Native ABI and lifetime handling is explicit. Those mechanisms justify code; removing them to meet a line-count target would degrade the product.

The dual preview design also serves two different needs: authoritative PDF output and interactive source navigation. The defect is coupling PDF success to optional rasterization, not the existence of two backends.

Finding register

ID	Level	Finding	Evidence
R01	P2	PDF export fails when optional rasterization fails	Reproduced
R02	P2	Save As / export stage on the source filesystem	Static
R03	P1	Gallery rollback deletes originals after partial failure	Reproduced
R04	P2	Capture mode can persist fixture settings/history	Static
R05	P2	Concurrent private-directory initialization races	Reproduced
R06	P2	Copied private mirror retains stale directory content	Reproduced
R07	P2	Transparent swatches multiply alpha twice	Reproduced
R08	P2	Identity theme transform changes appearance class	Static
R09	P2	Find recomputes expensive matches in the frame loop	Measured
R10	P2	Index scanner treats comments/raw samples as code	Reproduced
R11	P3	EditorApp coordinates too many state lifecycles	Architecture
R12	P3	Commands have several manually synchronized definitions	Architecture
R13	P3	Settings/font/theme application repeats policy	Architecture
R14	P3	Child-window chrome and ownership are repeated	Architecture
R15	P2	Worker failure handling and process cleanup are uneven	Static
R16	P2	Rendering performs avoidable source work and filesystem I/O	Static
R17	P2	Invalid CLI exits successfully; option conflict can panic	Mixed
R18	P3	Dead API wrappers and duplicated test/QA declarations	Call-site review

Recommended reading order

Pages 5–10 explain the correctness and performance findings. Pages 11–16 describe the abstractions and cleanup that would improve organization. Pages 17–19 give the reduction budget, rewrite decisions, implementation sequence, and verification record.

The findings are deliberately not all “delete this code.” Several correctness fixes add code. Their organizing abstractions make it possible to delete repeated coordination elsewhere, and give later changes a smaller place to live.

Separate artifacts from presentation

R01 / PDF success depends on Poppler success

P2 REPRODUCED

`render_pdf` reads the authoritative PDF bytes, then uses `?` on `rasterize_pdf`. If Poppler is absent or rasterization fails, the successful artifact is returned as an error. The app only stores PDF bytes and completes pending exports in the success branch. Requesting export enables this path even when the interactive preview works.

`src/compiler.rs:617–645`

`src/app.rs:1849–1887, 3685–3738, 4695–4715`

Observed: a probe invoking the actual function with an existing 20,138-byte PDF and `PATH=/var/empty` returned “Poppler was not found” instead of a successful artifact. This supports the failure mechanism without claiming a full GUI export reproduction. The recovery-only Poppler role is documented in `docs/architecture/0003-bundled-toolchain.md`.

Change: return `CompileArtifact { revision, pdf, diagnostics }` as soon as compilation succeeds. Submit a separate `RasterRequest` when page pixels are needed. Carry the same artifact identity through both paths, and let raster failure degrade the viewer while export remains available. Preserve byte-for-byte association of textures and artifacts; do not reread a changing watcher output later.

Verification: a successful compiler result with an intentionally missing rasterizer must remain exportable; raster recovery and stale-generation rejection must still work. This separation may initially increase LOC and should not be sold as a deletion-only change.

R02 / Atomic writes choose the wrong staging root

P2 HIGH-CONFIDENCE STATIC

`private_workspace::atomic_write` requires callers to choose the root from the destination so staging and destination are on the same filesystem. Both source saving and PDF export instead pass `self.project_root()`. Save As or Export to another mounted volume can therefore fail during `persist` even though the destination is writable.

`src/private_workspace.rs:237–261`

`src/app.rs:3267, 3722, 13344–13346`

Change: expose `AtomicFileWriter::write(destination, bytes)` and determine destination-local staging inside that API, preserving the repository’s private-artifact policy. Remove the invalid root/destination pair from ordinary call sites. Retain permission preservation, file sync, and parent-directory sync.

Verification: inject staging-root selection in deterministic tests; add a cross-volume integration check where available. No cross-volume mount was exercised in this audit. This is a failed-write issue, not evidence of an observed source-file corruption.

QA must preserve user state

R03 / Partial gallery backup loses original PNGs

P1 REPRODUCED WITH FAULT INJECTION

`backup_requested_outputs` moves original files one at a time, but enables restoration only after the whole loop succeeds. If the second move fails, cleanup skips restoration and deletes the backup directory containing the first original. Cleanup also ignores restoration errors before deleting whatever remains in that backup.

`scripts/capture-theme-gallery.sh:523–558`

Two temporary originals; exact current backup/cleanup functions; injected failure on the second mv:
Exit = 1 · first original exists = false · second original exists = true · backup exists = false.
No maintained gallery file was touched.

Change: stage newly generated outputs separately, validate them, and publish with an explicit transaction. If retaining per-file moves, track completed moves and failed restorations and retain the backup on any recovery failure. Setting the restoration flag earlier alone is unsafe: the existing restore loop first removes every destination, including originals not yet backed up.

Verification: behavioral tests must interrupt/fail every backup and restoration step and assert all original bytes survive. This is higher value than checking script text for a function name.

R04 / Capture mode bypasses the save guard

P2 HIGH-CONFIDENCE STATIC

`EditorApp::save` skips persistence when a snapshot scene is active. The production `AppShell::save` does not call that guard; it always writes merged settings. The shell obtains its initial settings after snapshot setup has reset font weights and remembered the fixture workspace. A capture run can therefore save QA defaults and fixture history during autosave or exit.

`src/app.rs:1101–1115, 9046–9053`

`src/windowing.rs:91, 373–377`

Change: give the shell an explicit `LaunchMode` and persistence policy. Interactive sessions may persist; deterministic capture sessions use transient settings. Make the outer owner decide, instead of relying on an inner trait implementation that the wrapper bypasses.

Verification: invoke shell persistence through an in-memory `eFrame::Storage` and assert that snapshot/batch runs write no settings. Real user preferences were not modified to reproduce this issue. A broader screenshot refactor must retain the one-session gallery and scene-to-viewport contracts.

Private workspace lifecycle

R05 / Concurrent initialization fails with AlreadyExists

P2 REPRODUCED

PrivateWorkspace::open checks whether .tiptotyp exists and then creates it. Two valid callers can both observe absence; one creates the directory and the other propagates AlreadyExists. Revalidation after creation protects path properties but does not make this ordinary concurrent initialization succeed.

src/private_workspace.rs:36–48, 264–274

A synchronized probe ran 8 opens against each of 100 fresh projects. In that run, 574 of 800 opens incorrectly failed, with “File exists (os error 17).” The rate is scheduling-dependent; the failure itself is the result.

Change: tolerate AlreadyExists from creation, then perform the existing symlink, directory, ownership-boundary, and permission checks. Centralize session directory ownership in a workspace service where useful, but retain validation even if callers share an object.

Verification: a barrier-based concurrent-open test must allow all valid callers, while existing rejection tests for symlinked or invalid private roots must still pass.

R06 / Fallback directory copies stop refreshing

P2 REPRODUCED

When symlinking is unavailable, the mirror copies a directory. On later refresh, mirror_entry sees an existing non-symlink destination, copies only when the source is a file, and otherwise returns success. Existing copied directories never receive nested changes or newly added files. The traversal also needs an explicit policy for removing stale copied entries.

src/private_workspace.rs:369–434, 436–457, 479–506

Probe of the actual mirror helpers after initial copy and source changes:
changed file = “old” · deleted file still exists = true · added file exists = false.

Change: represent mirror entries as linked versus copied, and give copied directories a complete synchronization policy. A fresh staged mirror per generation is simpler but may be expensive; an incremental copy synchronizer must handle add/change/delete and type changes. Choose based on measured project size, not a generic filesystem abstraction.

Verification: exercise forced-copy mode independently of host symlink privileges. Cover nested replacement/deletion and source type changes. The defect is established in the fallback code; this audit did not claim the default macOS symlink path always exhibits it.

One color model, one policy

R07 / Swatch composition applies alpha twice

P2 REPRODUCED

Color parsing creates `Color32::from_rgba_unmultiplied`, which stores premultiplied channels. `composite_over_editor` then multiplies `color.r()`, `g()`, and `b()` by alpha again. Transparent Typst color swatches are darker than intended.

`src/highlight.rs:640, 762–818, 1177–1192`

For a half-transparent red swatch over white, the exact helper returned `[191, 127, 127]`; its intended sRGB blend is `[255, 127, 127]`. The probe used the repository's actual ecolor dependency. The existing alpha test checks parsing, not compositing.

Change: do color math in a clearly unpremultiplied `Rgba` domain type, then convert to `egui` only at the display boundary. Test half-transparent foregrounds over light and dark backgrounds. A later pixel-changing fix also needs fresh visual evidence under `AGENTS.md`.

R08 / Identity changes light/dark classification

P2 HIGH-CONFIDENCE STATIC

The importer classifies dark backgrounds at relative luminance below 0.35. The transform reclassifies every result, including identity, at below 0.5. A `#aaaaaa` background has luminance about 0.402: it starts light and becomes dark without any color change. The app always applies this transform, and the result selects effective appearance and Typst override slots.

`src/sublime_theme.rs:412`

`src/theme_transform.rs:167–170, 231–242`

`src/app.rs:241, 2831–2838`

Change: share one appearance-classification policy and assert that an identity transform preserves the entire resolved theme. Non-identity transforms may intentionally change classification; that should use the same definition.

Consolidation opportunity

Hex parsing, HSL conversion, luminance, and alpha composition recur across `highlight.rs`, `sublime_theme.rs`, and `theme_transform.rs`. Move the math and domain types into `color / theme_model`; retain separate Typst and Sublime syntax adapters. `ImportedTheme` also represents built-ins, so a neutral `ResolvedTheme` name and home would express the actual role. Estimated net saving: **70–130 lines**, including these fixes rather than counting them again.

Search belongs to a document revision

R09 / Find repeatedly executes expensive matching

P2 MEASURED; FRAME-LOOP CALL PATH VERIFIED

`show_find_bar` materializes every match on each render just to obtain the count. Search navigation and replacement run matching again; regex mode rebuilds the pattern. Case-insensitive literal search also creates and extends a candidate string at every source character, retrying up to the query length even after an early mismatch could rule it out.

`src/app.rs:7438–7450`

`src/search.rs:63–100, 104–137, 153–192, 251–330`

Optimized isolated harness, actual `search.rs`, 50,000 a characters and a query of 999 as followed by b:
Case-sensitive: 169 microseconds. Case-insensitive: 366.8 milliseconds. Both returned zero matches.
These are one-machine probe timings, not application frame-rate measurements.

The nested candidate-building strategy scales with source length and query length, and the frame-loop caller makes repeated work visible even when neither the query nor the document has changed. The user's recent regex changes already use a bounded non-backtracking engine; this report does not attribute catastrophic regex backtracking to the current implementation.

Recommended abstraction

```
struct SearchSession {  
    key: SearchKey, // document identity, revision, query, options  
    query: CompiledQuery,  
    matches: Vec<SearchMatch>,  
    selected: Option<usize>,  
}
```

Counts, next/previous, and replace-one should consume this shared result. Replacement invalidates the revision and produces one new result. Regex parse errors should be distinct from a valid zero-match search; the current `.build` error path returns an empty vector.

For literal case-insensitive matching, fold text once and retain a map back to original byte and scalar boundaries. Preserve the current whole-character behavior and define Unicode semantics explicitly. Switching to regex case-insensitivity may change matching behavior, so it is not an automatic drop-in simplification.

Acceptance: count/navigation/replacement agree; invalid patterns surface as invalid; repeated rendering of an unchanged query performs no compilation or scan; zero-width matches and Unicode offsets remain correct. Estimated net LOC reduction is only **30–70**. The primary gain is predictable latency, not source compression.

Use the parser already in the repo

R10 / The project index has a second, weaker parser

P2 REPRODUCED

Headings, definitions, imports, and packages are recognized by a line scanner and hand-written helpers. Only references use the existing Typst AST. The scanner skips line comments but cannot distinguish multiline comments or raw code examples from active source.

src/project_index.rs:124–230, 232–281

The direct-module probe supplied a block comment containing a heading, a definition, and an include, plus a fenced Typst example. The index reported the commented and sample declarations and followed the commented include into `child.typ`. The scanner also has structural limitations around multiline imports and declarations inside code blocks; it examines only limited keyword occurrences per line.

Replace recognition with one AST visitor

Use `typst_syntax` to visit static headings, bindings, references, and literal `import / include` dependencies. Keep the unresolved status of dynamic paths explicit. Do not attempt to implement Typst evaluation in the project index.

```
struct ProjectIndexInput {
    root: PathBuf,
    entry: PathBuf,
    overrides: BTreeMap<PathBuf, DocumentSnapshot>,
}

// AST node kind decides whether a token is live source.
// Source position mapping supplies line numbers once.
```

Reference indexing currently counts preceding newlines separately for each reference. Reuse a source line index instead. For many references this removes repeated scans of the same prefix as well as the duplicate parser.

Acceptance: comments and raw examples contribute no active declarations or imports; multiline syntax is recognized; dependencies remain confined to the intended project policy; cyclic dependencies still terminate. Expected net production reduction: **60–120 lines**.

Extend the same ownership boundary carefully

`SyntaxHighlighter` already owns an incrementally updated `typst_syntax::Source`, while the app reparses detached sources for font argument and web-link queries. A document snapshot can own shared source metadata, allowing consumers to ask queries without reimplementing token recognition. The background indexer can own a revision-tagged snapshot rather than sharing mutable parser state across threads.

src/highlight.rs:15–35
src/app.rs:12872–12941

Give the session explicit owners

R11 / EditorApp is the integration boundary for everything

P3 ARCHITECTURE

The 145 fields in `EditorApp` include document identity, saved text, history, compile revisions, PDF bytes, page textures, project scans, font scans, 17 `applied_*` fields, dialogs, modal workflows, and several independent `Tinymist/webview` flags. Reset and restart paths must maintain relationships among them manually.

`src/app.rs:913–1070, 1730–1755, 2035–2067, 3911–3976, 4135–4304`

This is more than a long file: callers can mutate part of a conceptual state without owning its invariants. An `Option<Generation>`, readiness booleans, URIs, URL, and `ServiceState` can describe partially inconsistent service states unless every branch updates them correctly. Existing guards are valuable, but they are spread across the large owner.

Proposed decomposition

Owner	Responsibility and invariant
<code>DocumentSession</code>	Buffer, path, kind, saved snapshot, revision, dirty state, history. Every edit is one transaction.
<code>DocumentWorkflow</code>	Open/save/close/overwrite and dialogs; actions carry document identity and revision.
<code>PreviewController</code>	Requested/effective backend, artifact, raster requests, navigation, fallback state.
<code>WorkspaceModel</code>	Root, inventory, preview entry, index, refresh generations.
<code>PresentationState</code>	Resolved theme/font requests, applied configuration and UI state.
<code>EditorView</code>	Draws a stable view of state and emits typed actions.

Start with owned structs in the same crate. Move tests with the invariant they exercise. A separate crate per subsystem, trait for every service, or general event framework is unnecessary for the present application.

Use a small event reducer for the brittle workflows, with explicit effects such as `ReadFile`, `WriteFile`, `Compile`, or `ShowDialog`. Preserve existing document-epoch checks and the rule that a pending export can follow edits but cannot cross documents. Keep long-lived process state machines specialized rather than forcing them into a universal reducer.

LOC accounting: moving 8,000 lines out of `app.rs` deletes zero repository lines. Savings must come from collapsing duplicated transitions and derived fields after ownership is clear. Do not target a tiny coordinator by hiding the same mutable state behind forwarding methods.

Commands and applied settings

R12 / Command metadata is defined in several places

P3 ARCHITECTURE

The native menu has item descriptors and separately maintained command-to-tag and tag-to-command matches. The app adds overlapping `FileMenuAction` / `EditorMenuAction` types, keyboard dispatch, native dispatch, and egui popup definitions. Adding a command or shortcut requires synchronizing several representations.

`src/native_menu.rs:169–311, 467–521`

`src/app.rs:745–768, 2069–2315, 12515–12686`

Use: one typed `AppCommand` for common actions and a `CommandSpec` registry containing title, shortcut, menu placement, and native identifier. Native menus, egui menus, and shortcut routing consume descriptors. Compute enablement from the document view. Preserve precedence of modified shortcuts, text-widget editing behavior, platform menu conventions, and dynamic payload actions such as opening a specific link.

Expected net saving: 150–260 lines across native and app code, counted once. A descriptor table should remove duplicate definitions; an elaborate plugin command bus would add complexity without a current need.

R13 / Settings application maintains parallel field lists

P3 ARCHITECTURE

Font change detection compares many separate settings fields, then copies the same values into `applied_*` fields. Initialization and file pickers repeat UI/code font setup. The font loader separately selects faces from raw bytes and catalog metadata and rereads font files for multiple requested weights.

`src/app.rs:949–976, 1128–1154, 2666–2742, 3511–3606`

`src/theme.rs:207–285, 301–396, 481–498`

Use: comparable `FontSelection` and `ResolvedPresentationRequest` values, with one `AppliedPresentation` snapshot. Compare semantic subrequests to issue minimal font/theme/backend effects. Prepare font-family bytes and normalized face/axis metadata once, then derive the weight-specific egui entries.

Preserve collection face indices, fallback ordering, style/stretch ranking, and slider commit-on-release behavior. Apply settings before the frame as the current code does; this prevents mixed-style frames. Do not replace a precise subrequest comparison with “any setting changed, rebuild everything.”

Expected net saving: 100–180 lines in app settings plumbing plus **60–120** in font preparation. Sharing theme-role metadata and default palette derivation is a separate small opportunity, not a reason to remove the 32-theme catalogue.

Child windows and background work

R14 / Child views repeat host-level policy

P3 ARCHITECTURE

Settings, overrides, menus, modals, rename dialogs, and tooltips separately set up immediate viewports, styles/native themes, capture hooks, size constraints, close handling, and focus behavior. Some helper infrastructure already exists, but common host responsibilities remain mixed into each body.

`src/app.rs:5407–5802, 5904–6518, 11722–11919`

Use: a small `ChildViewHost` plus an explicit specification for owner viewport, role, sizing, decoration, capture target, and focus policy. Give the body a callback that returns an action. Share common mechanics while keeping **different** lifecycle rules for persistent settings windows, dismiss-on-blur menus, modal confirmation, and interactive tooltip handoff.

Estimated net saving: **250–450 lines**, with medium confidence until two representative view types are migrated. Keep pure geometry tests and fresh framebuffer evidence when pixels change. Native preview placement still needs bounds tracing and a separate observation of composition; do not treat a root framebuffer as proof of child-view geometry.

R15 / Failure ownership is inconsistent across workers

P2 HIGH-CONFIDENCE STATIC

`Session::spawn` creates the child and reader threads, then calls `send_initialize()`?. The `Session` itself has no `Drop` cleanup, so a write failure there bypasses the explicit `kill/wait/join` branches used earlier in startup. The outer sidecar owner cannot clean up a session that never reached it.

`src/tinymist.rs:834–932`

Separately, `project-index` spawning discards the spawn result. Its poll uses `try_recv().ok()`, so a disconnected channel remains “pending” and continues scheduling repaint. `Workspace` scanning handles disconnection differently; font scanning has yet another pattern.

`src/app.rs:1480–1537, 1604–1649, 3978–4027`

Use: an `RAII` process owner immediately after `spawn`, with explicit bounded shutdown and ownership transfer only when initialization succeeds. Use a small `LatestJob<T>` for one-shot scans, representing `idle/running/ready/failed` and distinguishing empty from disconnected channels. Report spawn errors instead of discarding them. Keep the compiler and `Tinymist`’s protocol-specific state machines separate.

Acceptance: startup-write failure leaves no child/readers; injected spawn failure clears the pending state; stale work cannot replace a newer result. Estimated consolidation: **50–100 lines** for process cleanup, plus **90–180** for scan/dialog polling. The latter includes the repeated file-dialog waker polling at app lines 3435, 3511, and 3608.

Related liveness gap: pending LSP requests have no deadline, and writes to child `stdin` block. A live sidecar that stops reading can stall the worker; joining that worker on `drop` can then stall shutdown (`tinymist.rs:742–750, 982–1017, 2343–2354`). Existing graceful-shutdown timeouts do not bound an already blocked write. Give the supervisor an independent termination path and test a fake server that neither reads nor exits. This hardening is likely to add code.

Make rendering consume prepared state

R16 / The frame loop repeats source and filesystem work

P2 STATIC COST PATHS; NO FULL-APP PROFILE

`show_editor` computes diagnostics, counts lines, scans for the longest line, and clones a complete editor snapshot before every rendered edit widget, even if the buffer does not change. Its custom history is capped at 100 entries, but each entry owns the whole `String`. This is bounded history, not an unbounded-memory leak.

`src/app.rs:5104–5111, 5130–5143, 7606–7649, 8112–8157`

Link/font queries construct detached `Typst` sources despite the highlighter already maintaining a parsed source. Explorer rendering compares paths by canonicalizing both inputs for active/preview checks on each visited node. Diagnostic targeting uses the same path helper. External-change detection reads and hashes the entire active file on the UI thread once a second.

`src/app.rs:4058–4081, 9580–9581, 11373–11378, 12914–12940`

Use versioned document and workspace views

Maintain a `DocumentSnapshot` with revision, source text, line starts, offset conversions, and parsed queries. Cache derived diagnostics and unwrapped-size inputs by document/diagnostic generation. Normalize filesystem identity during workspace scans and document-open transitions, and carry that identity into rendering. Revalidate at I/O boundaries instead of canonicalizing every row while painting. Do not use raw canonicalization as a universal path policy. A missing Save As destination, a symlinked project entry, an LSP URI, and an existing file identity have different requirements. Small named types and explicit conversion points are safer than a catch-all `normalize_path` helper.

Stage the editor work

First remove repeated unchanged-frame work and measure representative documents. For larger files, a delta-based edit transaction and history can replace full-buffer undo snapshots. A rope alone will not solve full-buffer `TextEdit` layout; a virtualized editor is a distinct product milestone and likely adds code initially.

Keep disk conflict checks authoritative when saving. A metadata-only external watcher can miss same-size/same-timestamp changes, so any polling optimization needs a deliberate content-validation policy. Avoid claiming a speedup from this audit beyond the measured search probe.

LOC expectation: shared coordinate/edit helpers may save **80–160 lines**, but caching and history improvements can add code. Deduplicate byte/scalar/UTF-16 conversions only behind explicit units and tests; these coordinates are not interchangeable.

Launch boundaries and dead code

R17 / CLI validation does not reliably fail the process

P2 REPRODUCED EXIT STATUS; STATIC PANIC PATH

main prints launch/capture configuration errors and returns `Ok(())`. Running the existing debug binary with an invalid snapshot scene produced an error message and exit status 0. A batch step followed by `--no-ui-screenshots` can also leave steps enabled conceptually while the controller is disabled; `AppShell::new` then expects a queued request and can panic.

```
src/main.rs:34-53
src/screenshot.rs:803-828, 887-923, 468-469
src/windowing.rs:93-96
```

Use: typed `LaunchOptions` in a launch module, normalized option spelling, cross-field validation, and a nonzero failure exit. Preserve non-UTF-8 file paths and `--`. Unknown flags should not silently become document paths. Test option ordering and conflicts without launching a native window.

Observed input: `target/debug/tiptotyp --ui-snapshot-scene=__invalid__`.

Static conflict: `--ui-screenshot-step catpuccin-latte,main,false,0 --no-ui-screenshots`.

R18 / Remove dead surfaces, not supported features

P3 CALL-SITE AND TEST-ORGANIZATION REVIEW

Candidate	Conclusion and action
<code>WorkspaceTree::new / refresh</code>	Production-unused wrappers with explicit suppression at <code>workspace.rs:112-115, 144-149</code> . Used by tests. Move to test helpers or gate them; only about 10-15 production lines.
<code>ThemeFormat::Builtin</code>	Live in application resolution (<code>app.rs:221</code>); its suppression supports standalone tests. Do not delete it as dead code.
<code>PreviewMode::Slide</code>	Explicitly unused at <code>tinymist.rs:94-105</code> ; the app only requests document mode. Remove this speculative variant unless slide support is scheduled.
Typst error branch	<code>compiler.rs:997-998</code> handles a command name never passed to this helper; the sole call at line 679 passes <code>pdftoppm</code> . Typst already has a separate path-aware error helper.
Manual test temp owner	<code>workspace.rs:235-273</code> duplicates ownership/cleanup already supplied by <code>tempfile</code> . Replace fixture plumbing; about 15-25 test lines.
Path-included modules	<code>tests/builtin_themes.rs:1-5</code> and <code>tests/sublime_theme.rs:1-2</code> compile production modules and their inline tests again. Expose a small library boundary or reorganize tests. This reduces redundant builds/executions, not production behavior.

Strict clippy passes on the current target. That is useful evidence against ordinary unused private code, but not proof of cross-platform reachability or meaningful use. No large safely deletable production subsystem was established. Palette data, test fixtures, native fallbacks, and platform-specific code should not be relabeled dead merely because they are large or inactive on this machine.

The target module boundaries

Boundary	Owns	Outputs
app/shell	Launch mode, persistence, active session, window requests	Session routing
document	Text, identity, revision, history, offset index	Snapshots / edit events
workflow	Save/open/close/dialog state and conflict checks	Typed effects
workspace	Inventory, entry point, index generation	Stable view data
build	Canonical artifact and compile state	Revision-tagged PDF
preview	Backend choice, pixels, geometry, native surface	Viewer state/actions
presentation	Resolved colors, fonts, applied requests	Styles / font resources
ui	Command adapters, child host, editor/settings views	Typed user intent
platform	Native handles, menu bridge, process ownership	Narrow platform APIs
qa / xtask	Scene metadata, gallery transaction, launch probes	Validated artifacts

A small API that removes coordination

```
struct DocumentKey { epoch: u64, revision: u64 }
struct CompileArtifact {
    key: DocumentKey,
    pdf: Arc<[u8]>,
    diagnostics: Vec<Diagnostic>,
}

enum PreviewEvent {
    Compiled(CompileArtifact),
    RasterReady { key: DocumentKey, pages: Vec<PreviewPage> },
    RasterFailed { key: DocumentKey, message: String },
}
```

This is an illustrative boundary, not a proposed drop-in patch. It makes artifact success independent of rendering, preserves exact identity, and gives export a direct dependency on the artifact. Use similarly small types for applied font requests and pending document actions.

Consolidate metadata where repetition is declarative

Syntax roles repeat across enum, labels, groups, samples, and tag mappings (syntax_theme.rs:28–216, 446–468). A local descriptor declaration can generate these projections while keeping upstream tag matching exhaustive.

Scene/theme/filename metadata is repeated in Rust, Bash, and tests (screenshot.rs:69–244, 1151–1176; gallery script lines 70–104, 165–251). Let one contract produce a manifest for the runner. Keep a deliberately maintained gallery subset and golden filename expectations; do not accidentally expand the stable gallery to every possible combination.

What can actually reduce LOC?

The table is a **planning envelope**, not a measured patch. Each row describes net production/tooling lines after replacement scaffolding; tests remain outside the savings target. Moving code is counted as zero. Lower and upper ends assume a successful, appropriately narrow implementation.

Consolidation	Net lines	Primary reason
Commands and metadata	150–260	One definition for adapters
Child-view hosting	250–450	Shared native/capture/focus mechanics
Applied settings requests	100–180	Remove parallel field plumbing
Scan and dialog polling	90–180	Consistent state and error handling
Color math and classification	70–130	One representation and policy
Project AST visitor	60–120	Delete line-based recognizers
Search session	30–70	Share match results and navigation
Font-family preparation	60–120	One face-selection implementation
Syntax role descriptors	70–120	Generate repeated projections
RAII process cleanup	50–100	One cleanup owner
Gallery contract / runner	180–350	Remove cross-language duplication

The arithmetic range is 1,110–2,080 lines, rounded to **1,100–2,100**. This is about **3.7–6.9%** of the 30,025-line measured production/tooling proxy: 29,328 Rust lines before inline tests plus the 697-line gallery script. It excludes uncertain cache/history rewrites, palette fallback consolidation, and speculative path removals. Command and gallery rows already include their overlapping app/parser work.

That range is plausible enough to guide small spikes, not reliable enough to promise delivery. New artifact types, cancellation/error handling, or a library seam may absorb some savings. Recalculate against actual patches after the first two migrations.

How to evaluate a reduction

- Name the duplicate behavior being deleted and show that one owner now enforces it.
- Retain acceptance tests and public behavior; do not count moving literals to data files as eliminating complexity.
- Report repository-wide before/after physical lines, with tests and generated/data files separated.
- Prefer fewer places to change a feature over a lower raw line count produced by macros or compressed formatting.

A 30–50% reduction of the full repository is not supported by this review while preserving features and coverage. Achieving that would require explicit feature removal or a substantially different product boundary.

Refactor, rewrite, or keep?

Decision	Rationale
Refactor the app	Extract owned session components and views incrementally. Preserve working UI and native integration; avoid a second implementation of the entire product.
Rewrite the index scanner	A bounded replacement with the existing Typst AST removes faulty recognition logic and is easy to verify against fixtures.
Rewrite literal search internals	The measured candidate-building cost warrants a linearized matching design and revision cache. Preserve explicit Unicode semantics.
Consider a Rust gallery runner	The existing script is 697 lines and spans validation, processes, transactions, and image checks. A typed runner can reuse scene metadata and error handling; spike the LOC/capability tradeoff first.
Keep the dual preview	Interactive source mapping and canonical PDF serve different purposes. Decouple their success/failure contracts.
Keep theme catalogue data	Recipes already compress related palettes. Externalizing data changes line accounting and adds schema work; it is not a large deletion opportunity.
Defer virtualized editor	First remove repeated frame work and profile. A rope-backed/virtualized editor is a performance milestone, not a guaranteed LOC reduction.
Keep explicit native adapters	FFI lifetimes, focus handoff, clipping and platform distinctions need visible contracts. Avoid a generic platform framework.

Smaller worthwhile improvements

Font discovery and workspace scanning independently traverse files, and the app walks the snapshot again to collect font paths. A shared `WorkspaceInventory` with explicit visibility policies can feed font preparation without rescanning (`workspace.rs:153–213`; `font_catalog.rs:352–384`; `app.rs:12753–12769`). System-font discovery remains separate.

The fallback palettes in `theme.rs:1173–1277` overlap the built-in defaults but are not identical. Choose one supported fallback contract before consolidating. Thread-local active palette state (`theme.rs:23–36`) is a hidden dependency; make resolved presentation state explicit where practical, accepting that plumbing may add lines.

Version and package layout metadata also has multiple sources: runtime constants, the sidecar manifest, and an embedded TinyMist license URL in `xtask`. A small shared metadata model would prevent drift. Resource-style fallback layouts in `toolchain.rs:266–288` have no maintained producer found, but are not proven unreachable; confirm support before removal. These are lower-priority opportunities than the demonstrated defects.

Greenfield policy favors removing obsolete names or unsupported branches when confirmed. It does not justify deleting working fallbacks or introducing migration aliases during this refactor.

Execution plan and verification record

Sequence the work in reviewable changes

1. Repair the failure boundaries. Fix gallery transaction recovery, shell capture persistence, concurrent private-root creation, invalid launch exits, copied-mirror synchronization, and artifact/raster separation. Add direct boundary tests before broad movement of code. Implement color fixes with the required fresh visual QA.

2. Remove measured repeated work. Introduce `SearchSession`, replace the project line scanner with an AST visitor, and cache revision-derived editor data. Measure search, unchanged-frame source work, and representative large documents. Keep correctness and behavior tests as the acceptance criteria.

3. Establish owners and consolidate. Extract document/workflow and preview controllers, then command descriptors, applied presentation requests, worker ownership, and the child-view host. Migrate two representative consumers before generalizing. Publish the actual LOC result for each step.

4. Simplify infrastructure. Share the capture manifest, replace text-spelling tests with a stub-process gallery harness, expose a small library API for integration tests, and remove test-only production wrappers. Regenerate and validate the stable gallery when its machinery or visual contract changes.

Required boundary tests currently missing from the passing checks

- Successful PDF export with the rasterizer absent; stale raster results after a newer artifact.
- Partial backup and failed restoration with all original gallery bytes preserved.
- Capture-mode shell persistence through in-memory storage; CLI conflicts before GUI startup.
- Concurrent private-root opens and copied-mirror add/change/delete/type changes.
- Transparent swatch composition and identity-preserving theme classification.
- Comments/raw samples in indexing; search count/navigation/cache agreement.
- Tinymist initialization-write failure and one-shot worker disconnection.

Reproduction and measurement record

The search and project-index probes imported the actual repository modules. The alpha probe used the exact helper and current dependency. Private-workspace and PDF probes used temporary module copies with narrow probe entry points; application source was unchanged. The gallery probe extracted the exact transaction functions and injected a second-move failure against temporary files. Timing and race-frequency figures describe those single audit runs.

The attached evidence files retain file hashes, LOC counts, Cargo check output, and probe sources where available. Rebuild the report from the repository root with:

```
typst compile output/pdf/repository-audit.typ \
  output/pdf/repository-audit.pdf
```

Typst 0.15.0 compiled this report. Application validation used the repository's configured Rust toolchain. The report recommends changes; it does not claim that any finding has already been fixed.